

Paso 1

Cloné el repo de mexi-banco con el comando git clone a una carpeta donde tengo todo lo del curso. Con el clone también se imprime un mensaje explicando cómo cambiar de ramas para experimentar con el repo.

```
((base) alex@Apples-MacBook-Pro sistemasDistribuidos % git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 537.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

((base) alex@Apples-MacBook-Pro sistemasDistribuidos % cd mexi-banco
(base) alex@Apples-MacBook-Pro mexi-banco %
```

Paso 2

Para crear un contenedor usé el comando docker compose up --build -d. El parámetro --build hace que se ejecute el contenedor y el parámetro -d hace que dicha ejecución se haga en segundo plano.

```
((base) alex@Apples-MacBook-Pro mexi-banco % docker compose up --build -d
[+] Building 166.5s (18/18) FINISHED                                docker:desktop-linux
=> [app internal] load build definition from Dockerfile              0.0s

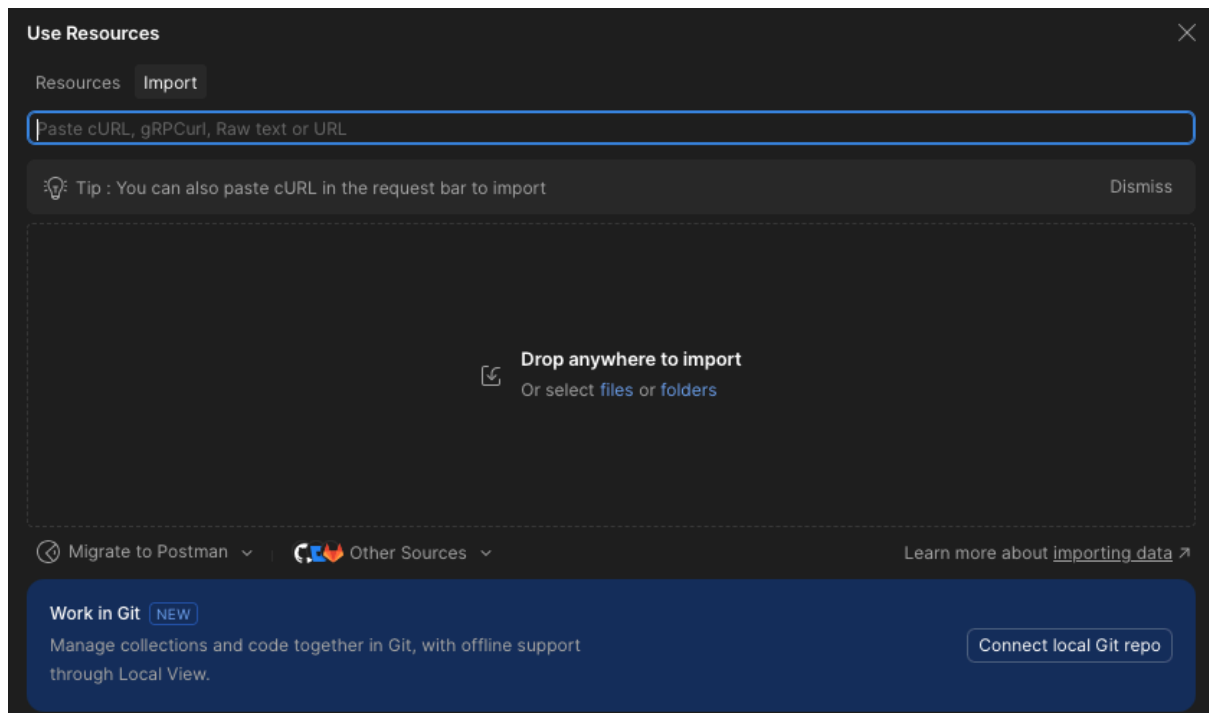
[+] Running 5/5
✔ app                               Built
✔ Network mexibanco                 Created
✔ Volume "mexi-banco_mexibanco_datos" Created
✔ Container mexi-banco-db-1          Healthy
✔ Container mexi-banco-app-1         Started
```

Luego utilicé docker compose ps para verificar que el contenedor estuviera en estado healthy. Aquí también se puede ver que hay un servicio de aplicación, uno de base de datos y qué puertos expone cada uno. La aplicación está construida con java y la base de datos es de postgres.

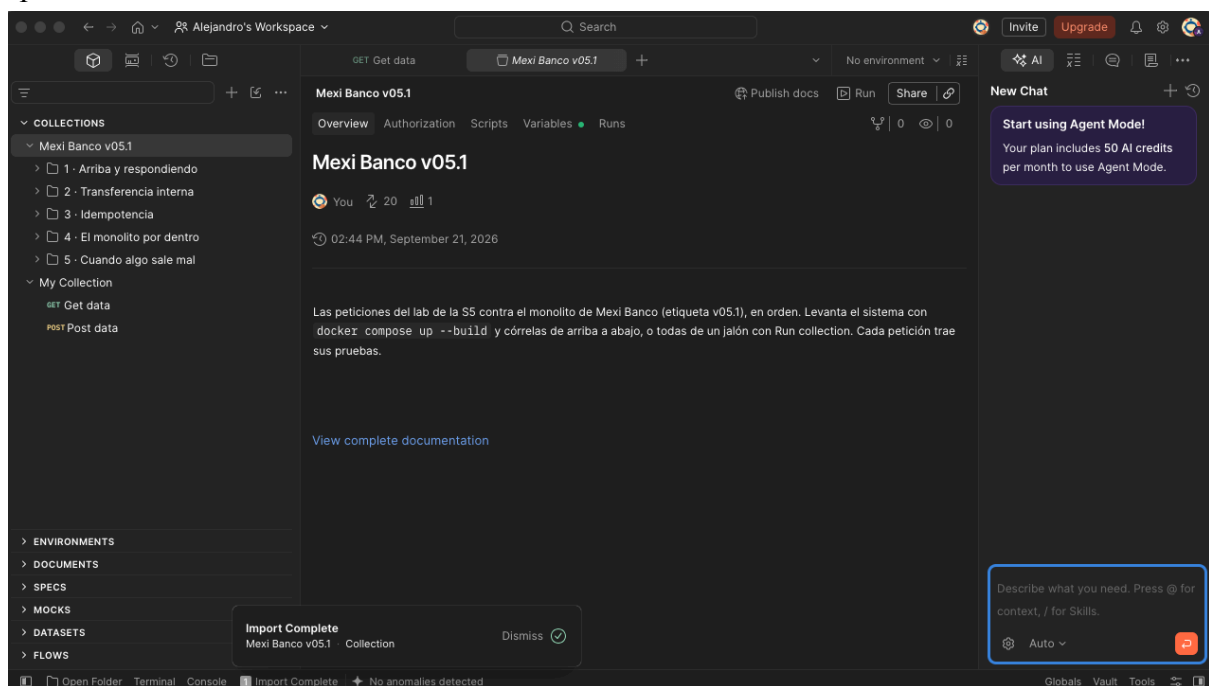
```
((base) alex@Apples-MacBook-Pro mexi-banco % docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS              PORTS
mexi-banco-app-1    mexi-banco-app       "java -jar app.jar"    app        4 minutes ago  Up 4 minutes (healthy)  0.0.0.0:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine   "docker-entrypoint.s..." db         4 minutes ago  Up 4 minutes (healthy)  5432/tcp
```

Paso 3

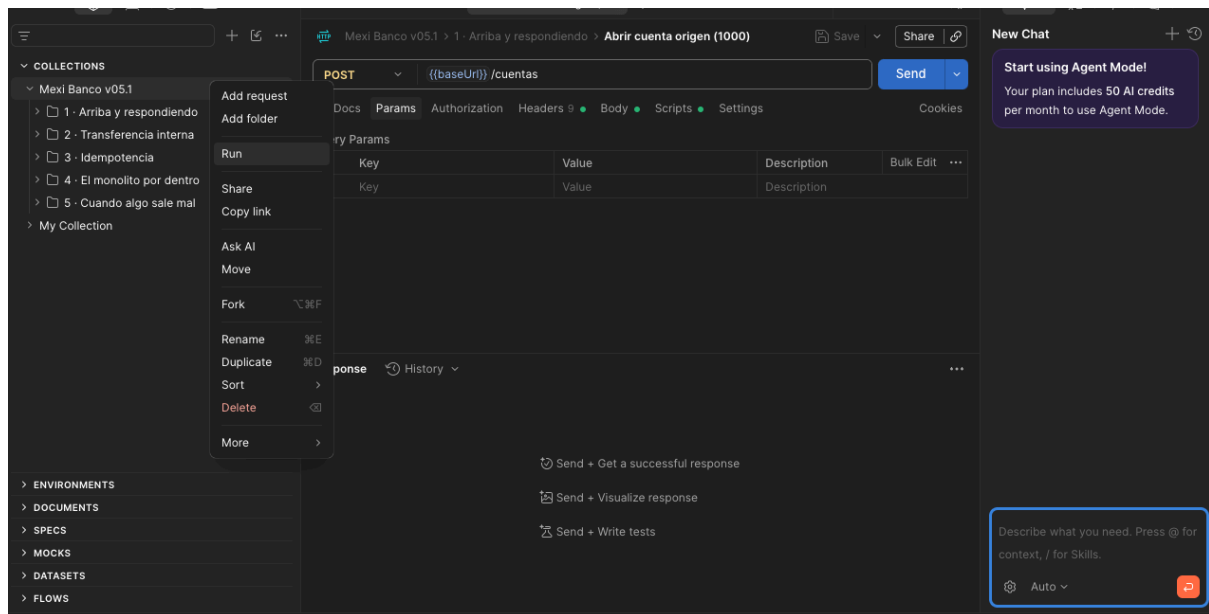
Posteriormente instalé Postman y coloqué la colección de mexi-banco en la pestaña de import.



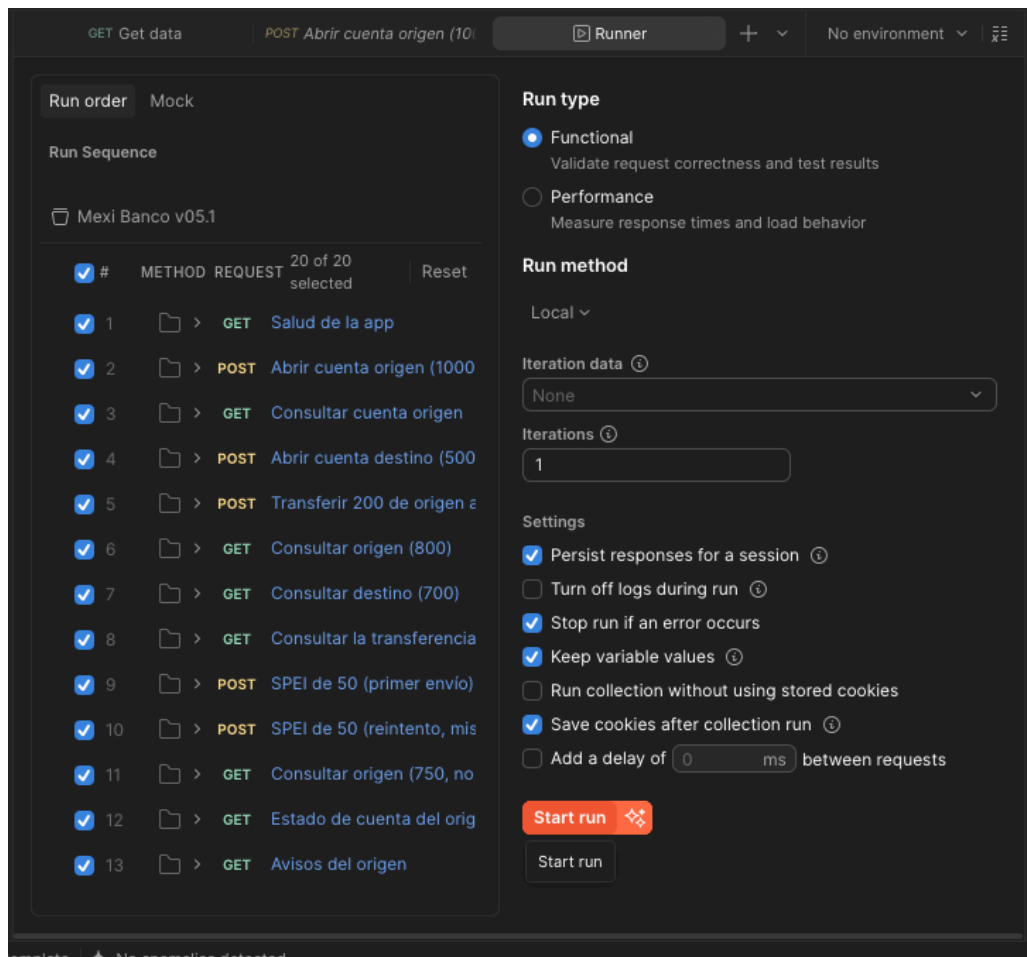
Una vez cargada la colección de mexi-banco se pueden ver 5 módulos con distintas operaciones.



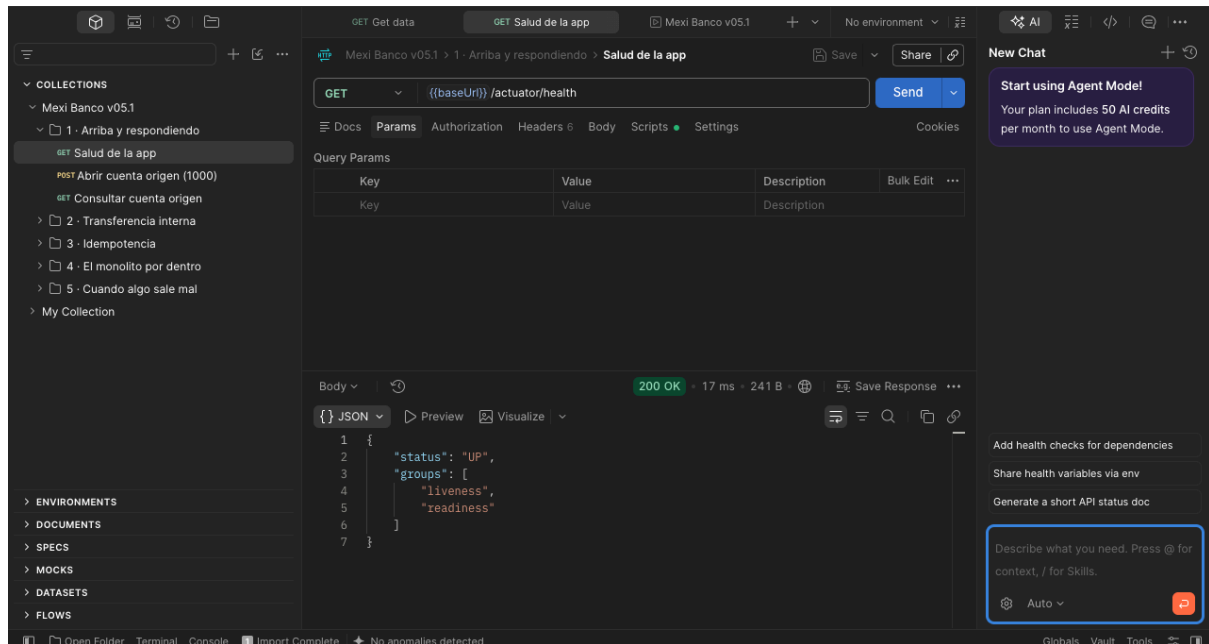
Probé la función de run para ejecutar todas las operaciones de cada módulo.



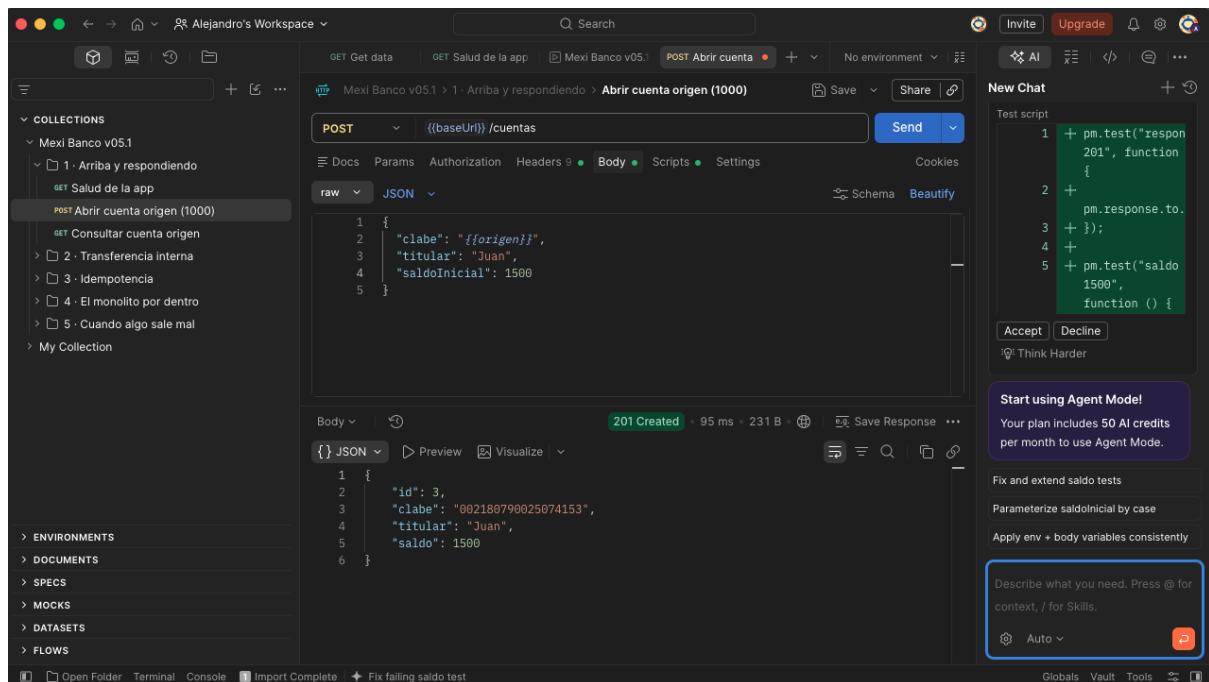
Al terminar de ejecutarse obtuvo los mismos resultados vistos en clase.



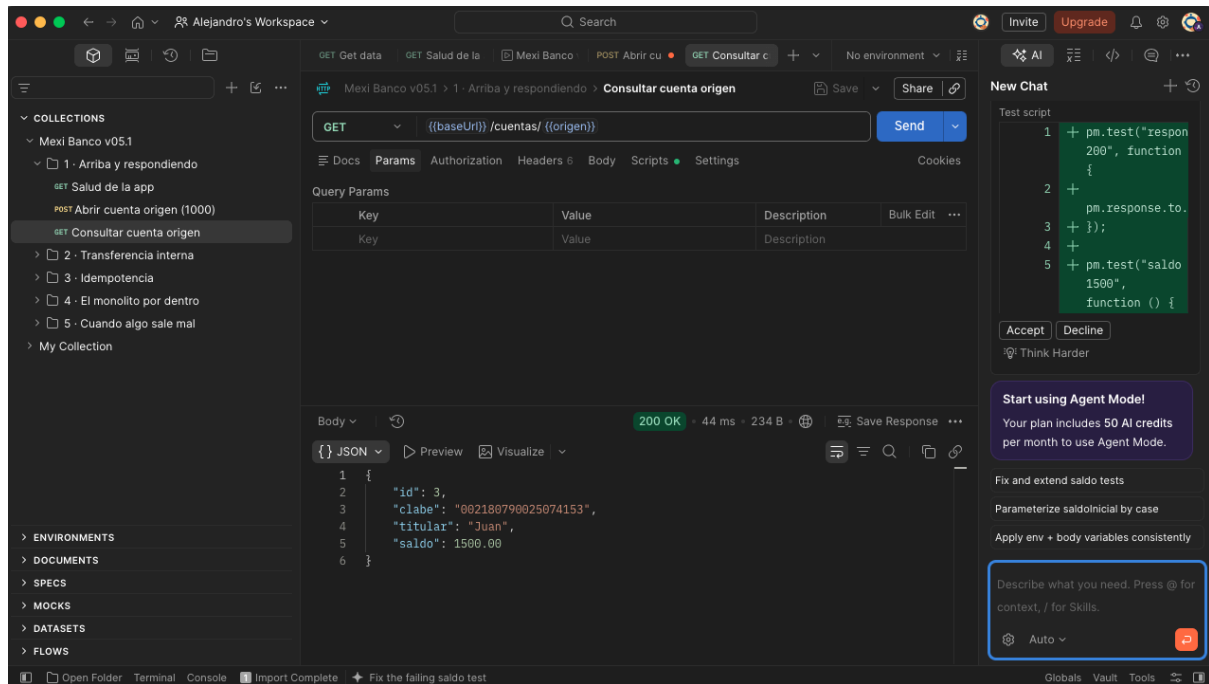
Después de ejecutar todo con run empecé a mandar peticiones una por una y cambiando algunos valores para crear distintos usuarios. Primero utilicé la solicitud de salud de la app donde me devolvieron la siguiente información.



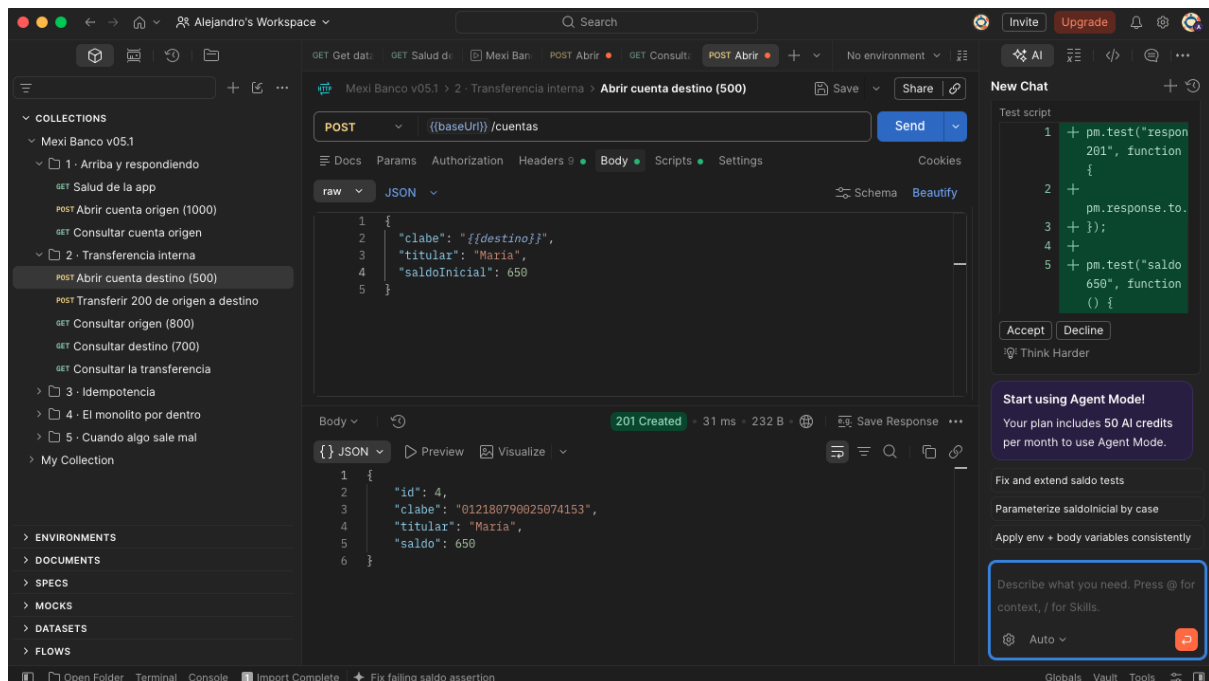
En esta captura envié una petición para crear una cuenta nueva para el titular Juan con un saldo inicial de 1500.



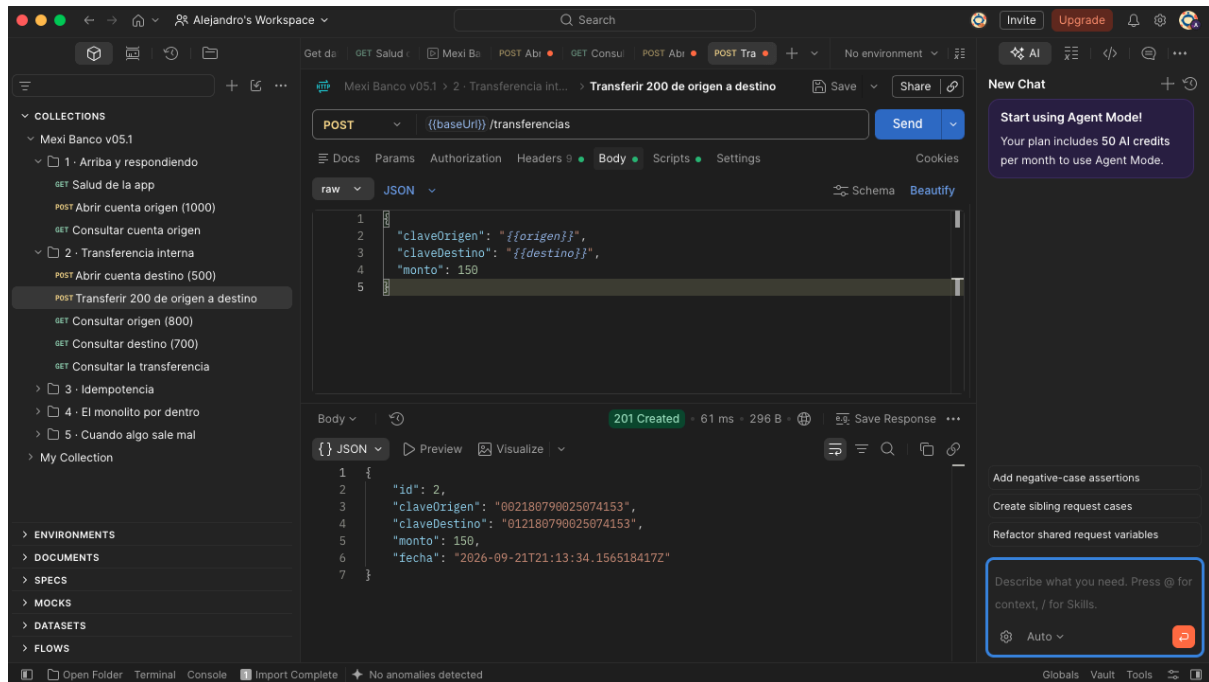
Luego consulté la cuenta origen de Juan y obtuve los siguientes datos.



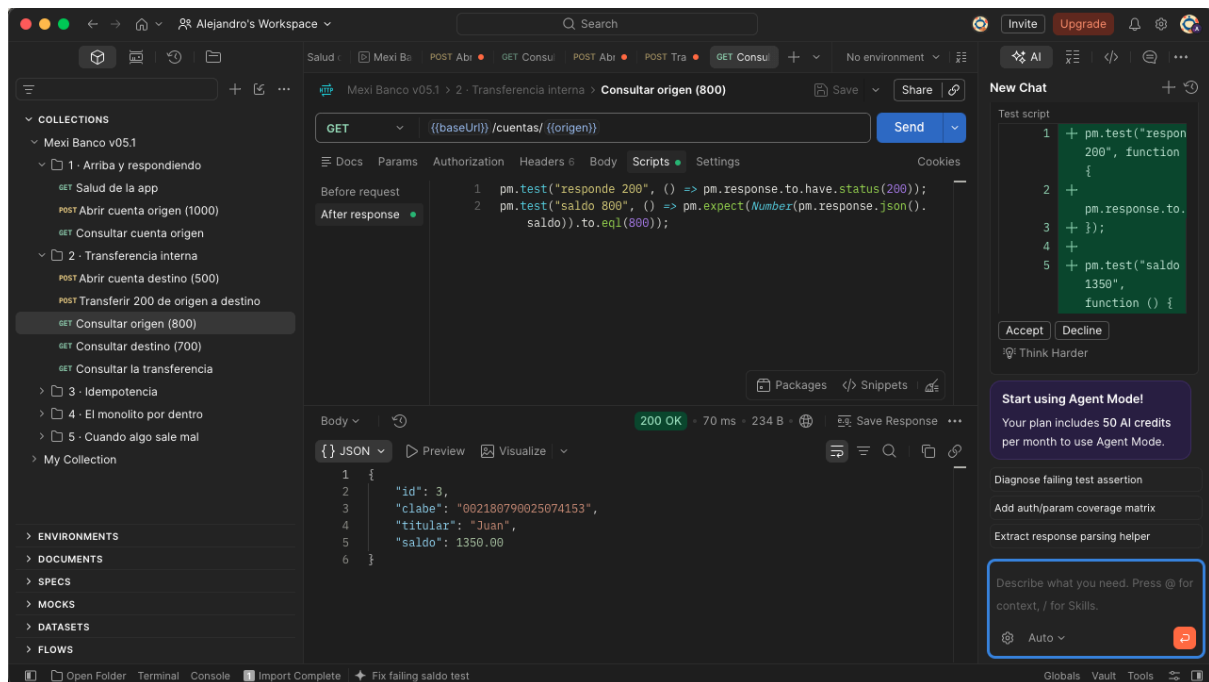
Pasando al siguiente módulo creé la cuenta de destino de María con un saldo inicial de 650.



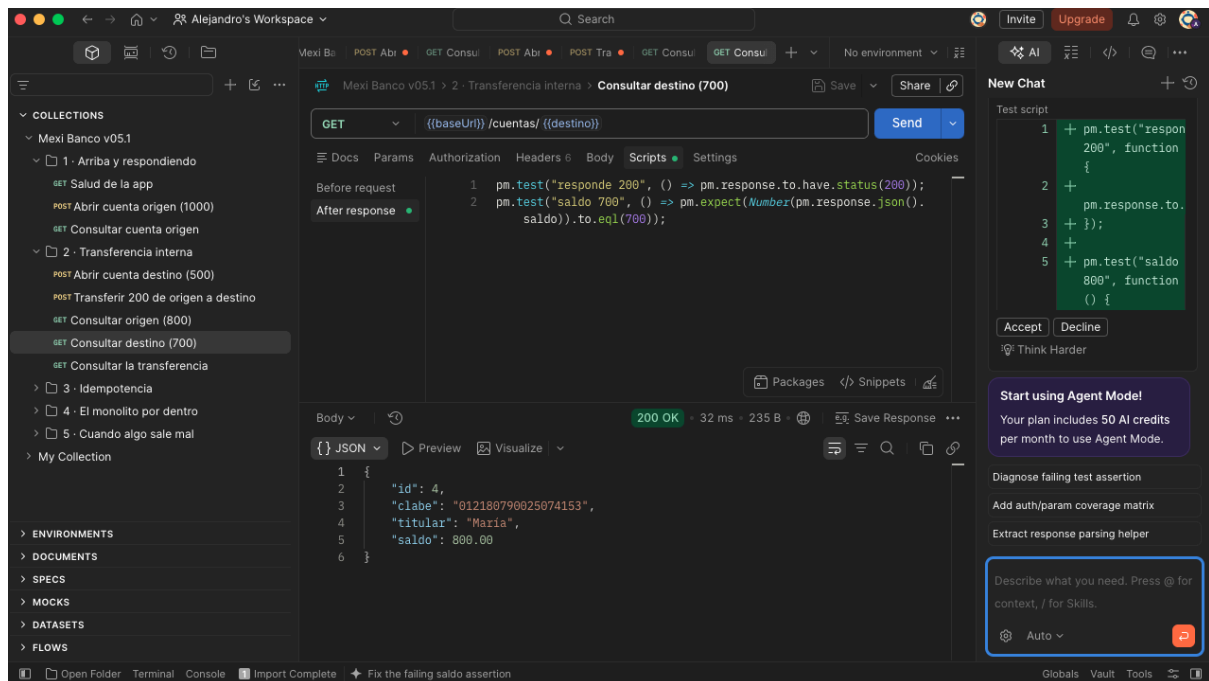
Luego hice una transferencia de la cuenta de Juan a la cuenta de María por un monto de 150.



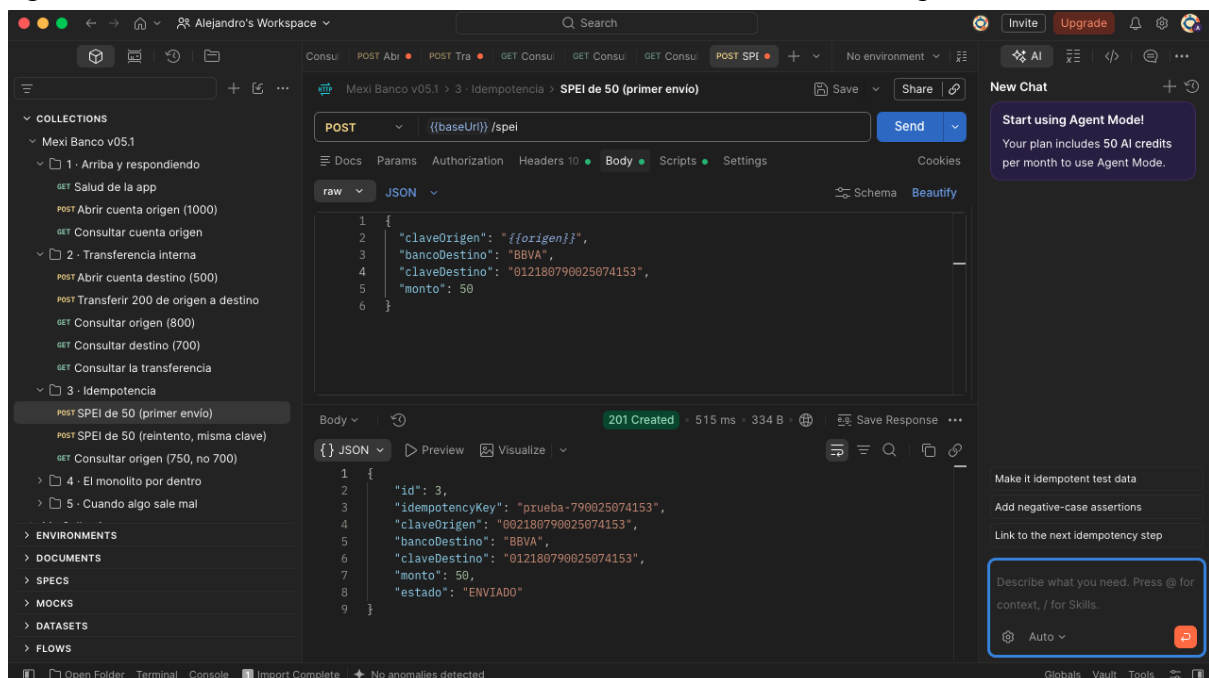
Al terminar la transferencia consulté la cuenta de Juan y se le restaron 150 a su saldo inicial de 1500, dejando un resultado de 1350.



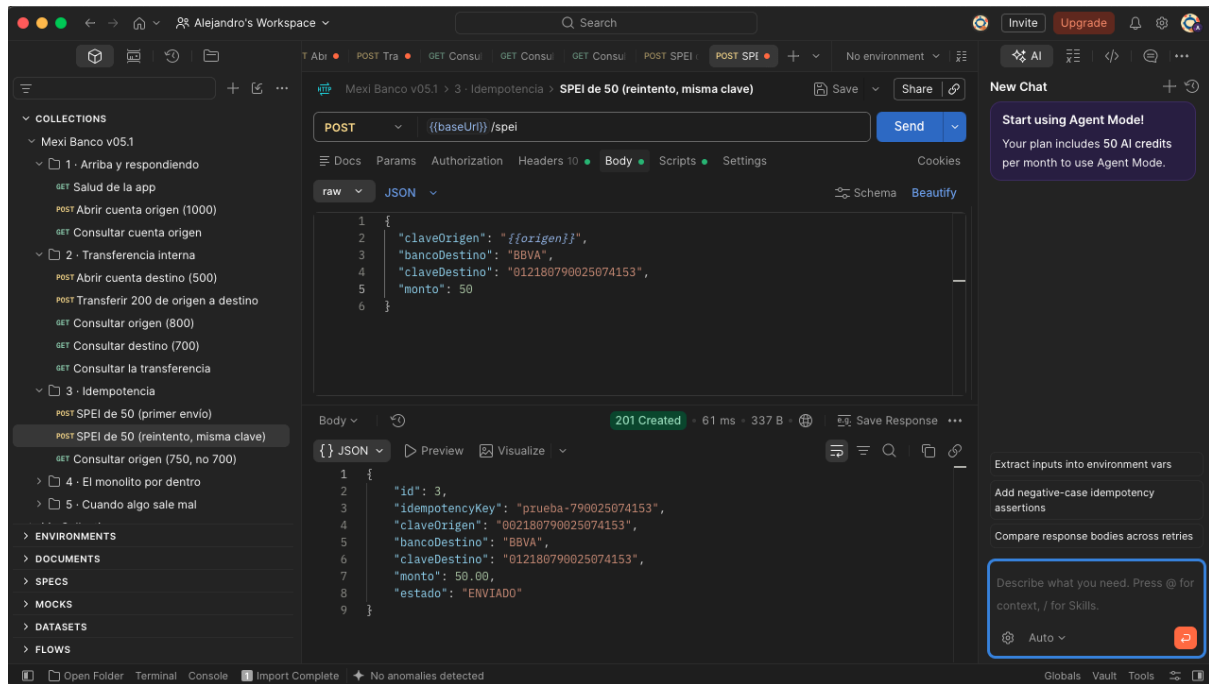
Luego consulté la cuenta de María y se le sumaron 150 a su saldo inicial, dando como resultado 800.



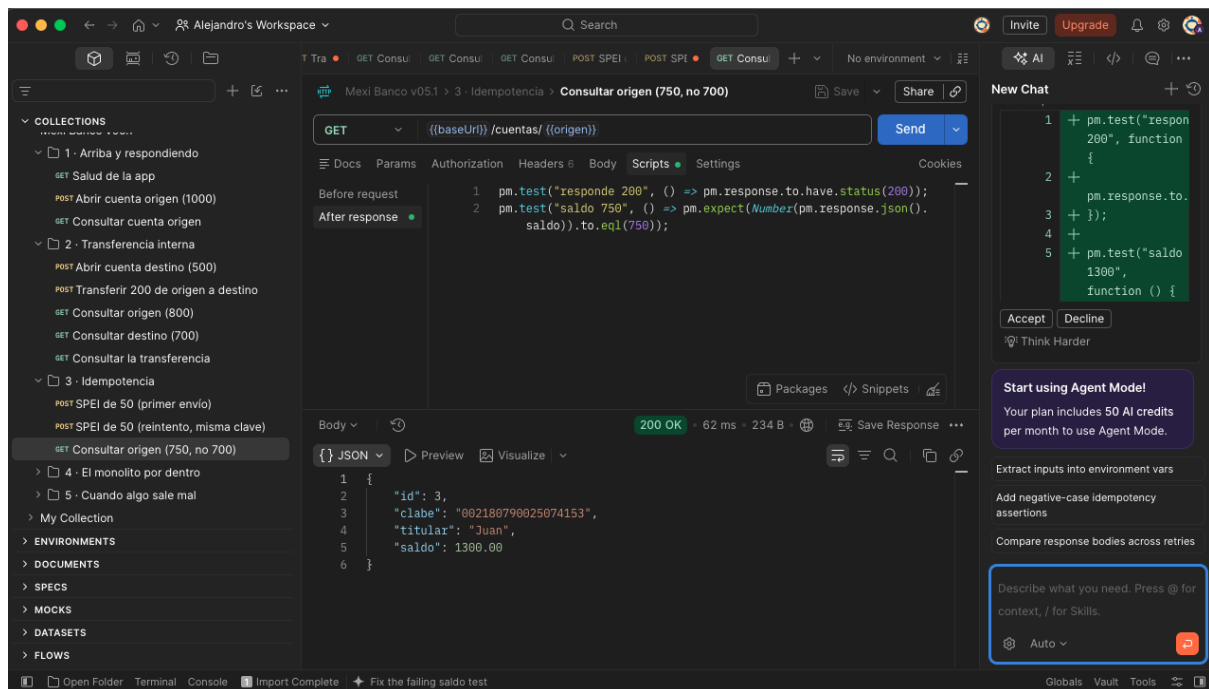
Posteriormente consulté la transferencia que se hizo y me imprimieron el id de la transferencia, las claves origen y destino, el monto y la fecha en que se realizó. Pasando al siguiente módulo se hizo un SPEI de la cuenta de Juan al banco BBVA por un monto de 50.



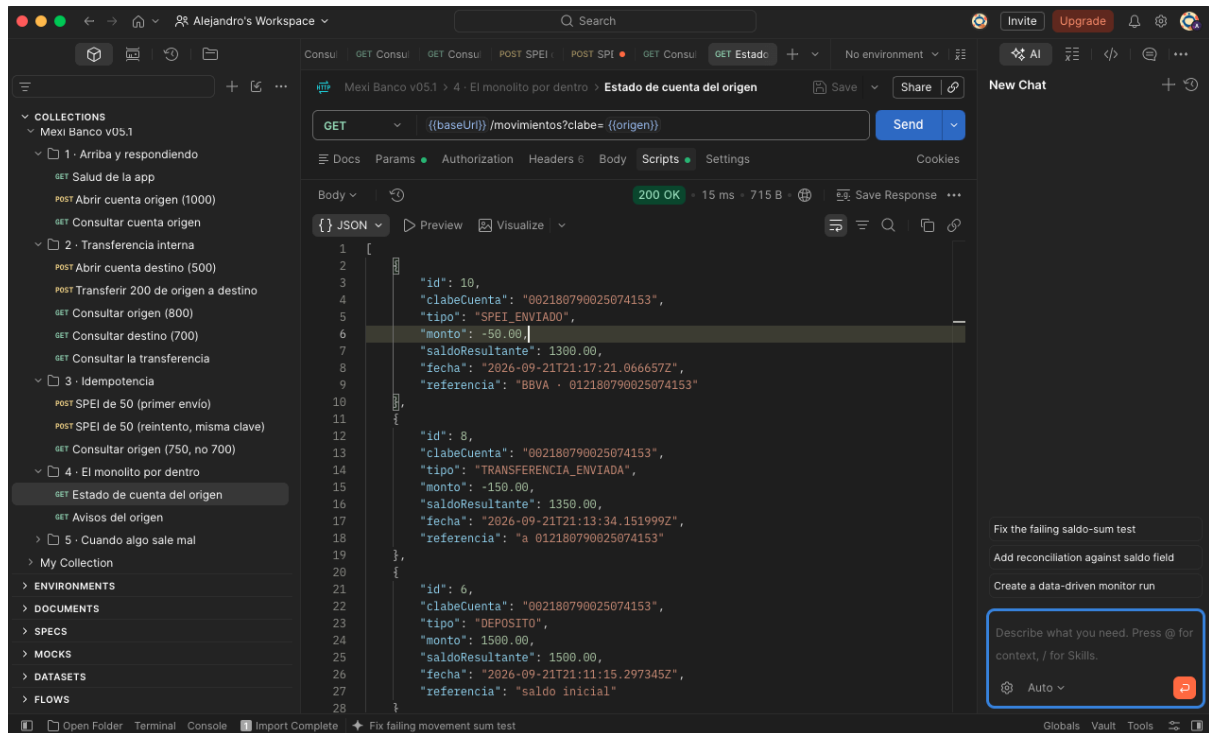
Luego se vuelve a hacer el mismo SPEI, donde se obtiene el mismo id de operación y la misma clave de idempotencia.



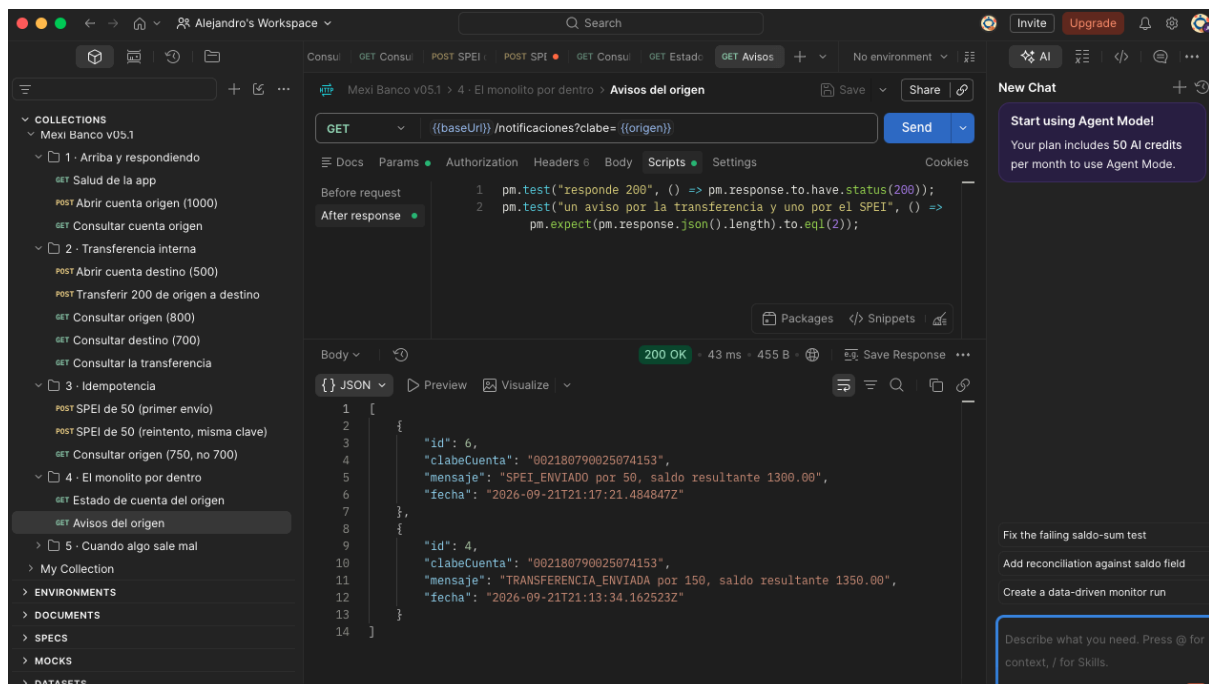
Ahora se consulta la cuenta de Juan y se puede ver que solo se le restó 50 a su saldo anterior de 1350, resultando en 1300. Lo cual quiere decir que solo se aplicó la operación una vez.



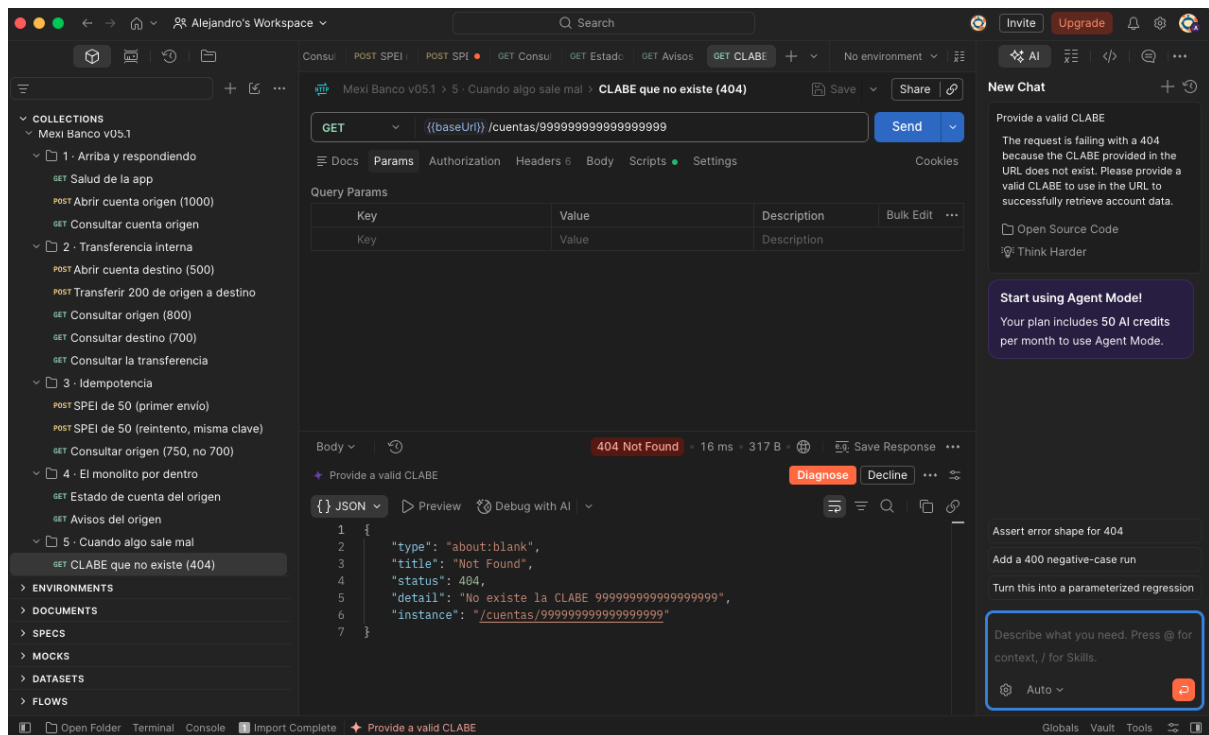
Luego se hizo la primera petición del 4to módulo donde se consultan las operaciones que se han hecho con la cuenta de origen hasta el momento mientras el contenedor ha estado ejecutándose. Se recalca que el SPEI por 50 solo salió una vez.



El siguiente módulo muestra los mensajes que recibiría Juan después de realizar la transferencia y el SPEI.



Con la petición de último módulo se intenta consultar una cuenta que no existe y se obtiene el siguiente resultado.

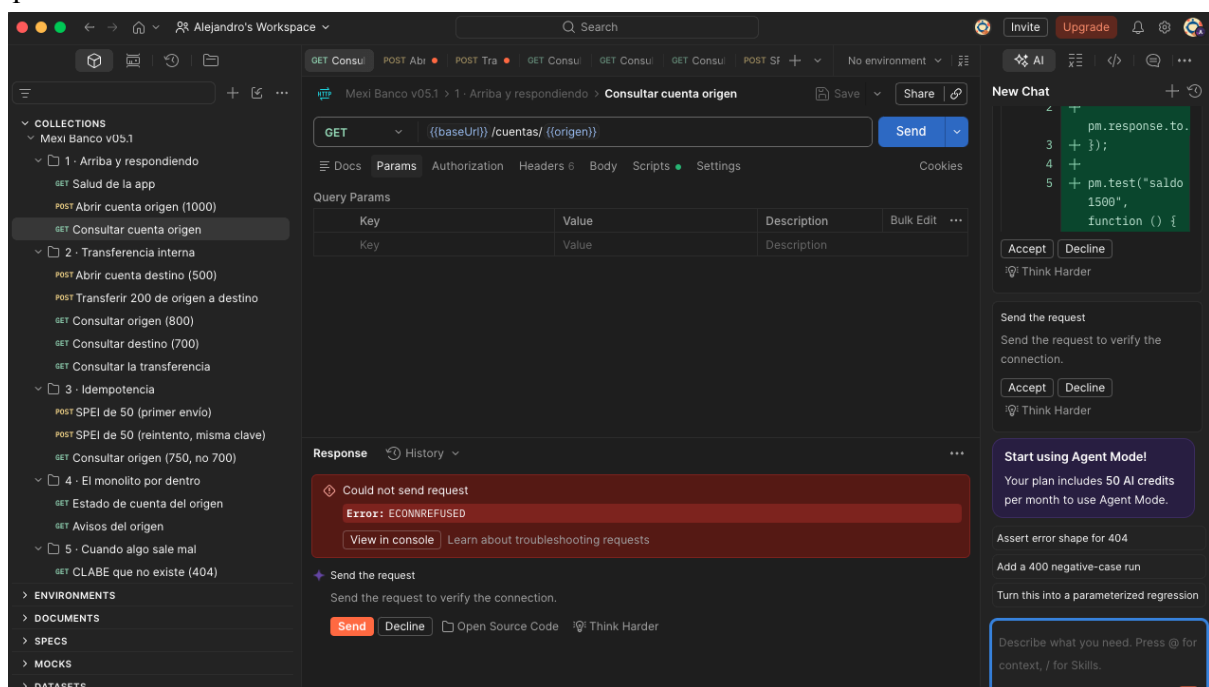


Paso 4

Ahora se usa `docker compose down` se eliminan los servicios y la red levantada con `docker compose up`, a excepción del volumen.

```
(base) alex@Apples-MacBook-Pro mexi-banco % docker compose down
[+] Running 3/3
 ✓ Container mexi-banco-app-1 Removed 0.4s
 ✓ Container mexi-banco-db-1 Removed 0.6s
 ✓ Network mexibanco Removed 0.3s
(base) alex@Apples-MacBook-Pro mexi-banco %
```

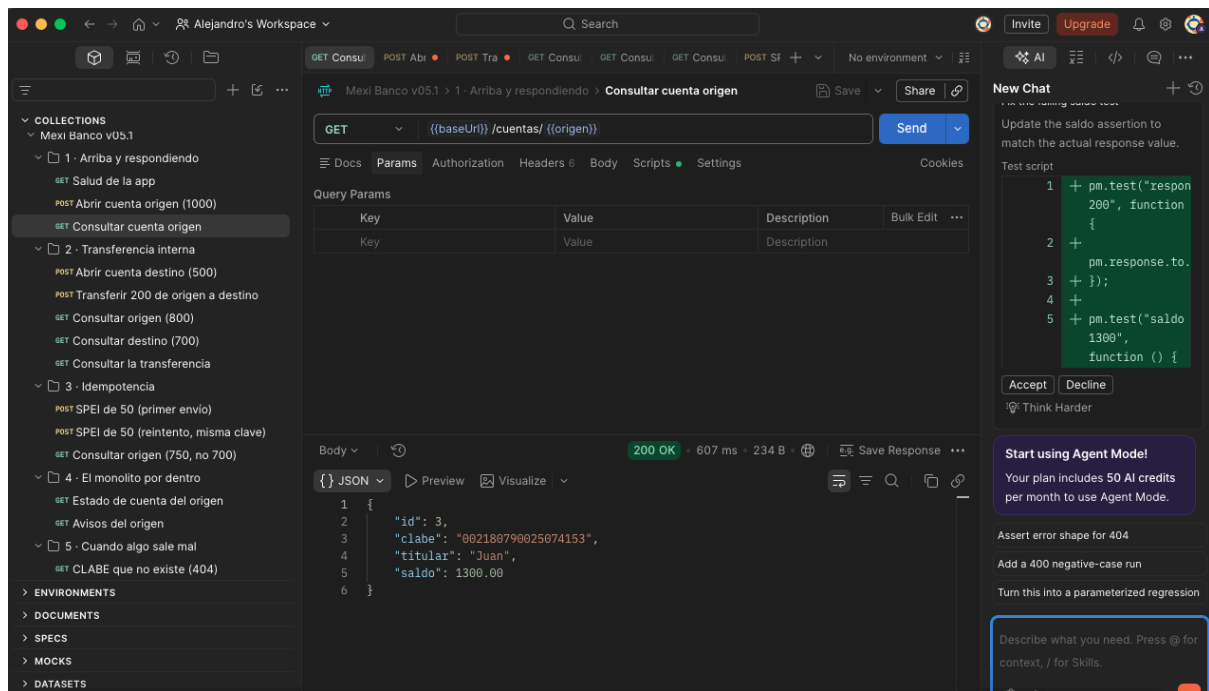
En este estado, al enviar una solicitud se devuelve un mensaje de error porque no hay nada que la reciba.



Se vuelve a usar `docker compose up -d` para que se ejecute el contenedor con la aplicación y la base de datos en segundo plano.

```
((base) alex@Apples-MacBook-Pro mexi-banco % docker compose up -d
[+] Running 3/3
 ✓ Network mexibanco      Created
 ✓ Container mexi-banco-db-1 Healthy
 ✓ Container mexi-banco-app-1 Started
```

Debido a que el volumen hace que persistan los datos, al consultar la cuenta de origen se puede ver la información de la cuenta de Juan creada en el paso anterior.



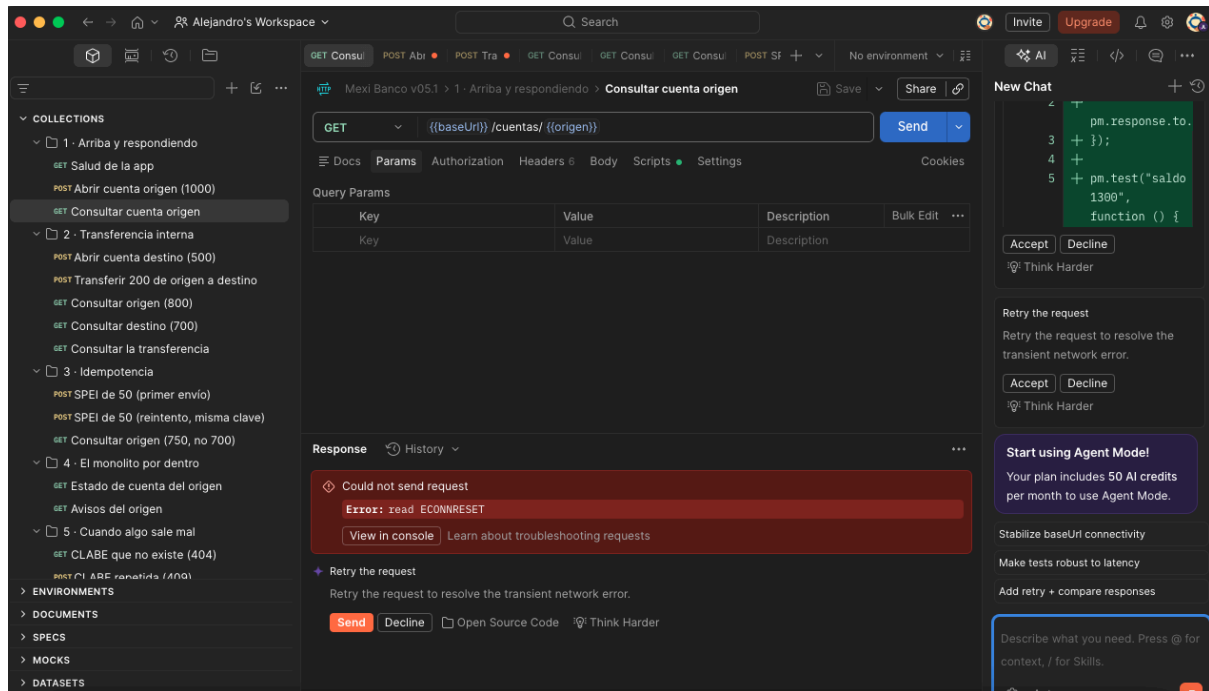
Ahora uso `docker compose down -v` para eliminar también el volumen.

```
((base) alex@Apples-MacBook-Pro mexi-banco % docker compose down -v
[+] Running 4/4
 ✓ Container mexi-banco-app-1 Removed
 ✓ Container mexi-banco-db-1 Removed
 ✓ Volume mexi-banco_mexibanco_datos Removed
 ✓ Network mexibanco Removed
```

Vuelvo a crear el contenedor y ejecutarlo en segundo plano.

```
((base) alex@Apples-MacBook-Pro mexi-banco % docker compose up -d
[+] Running 4/4
 ✓ Network mexibanco      Created
 ✓ Volume "mexi-banco_mexibanco_datos" Created
 ✓ Container mexi-banco-db-1 Healthy
 ✓ Container mexi-banco-app-1 Started
```

Ahora si consulto la cuenta de origen me devuelve un mensaje de error porque no existe ninguna cuenta de origen. Al eliminar el volumen ya no persistieron las cuentas ni las transacciones.

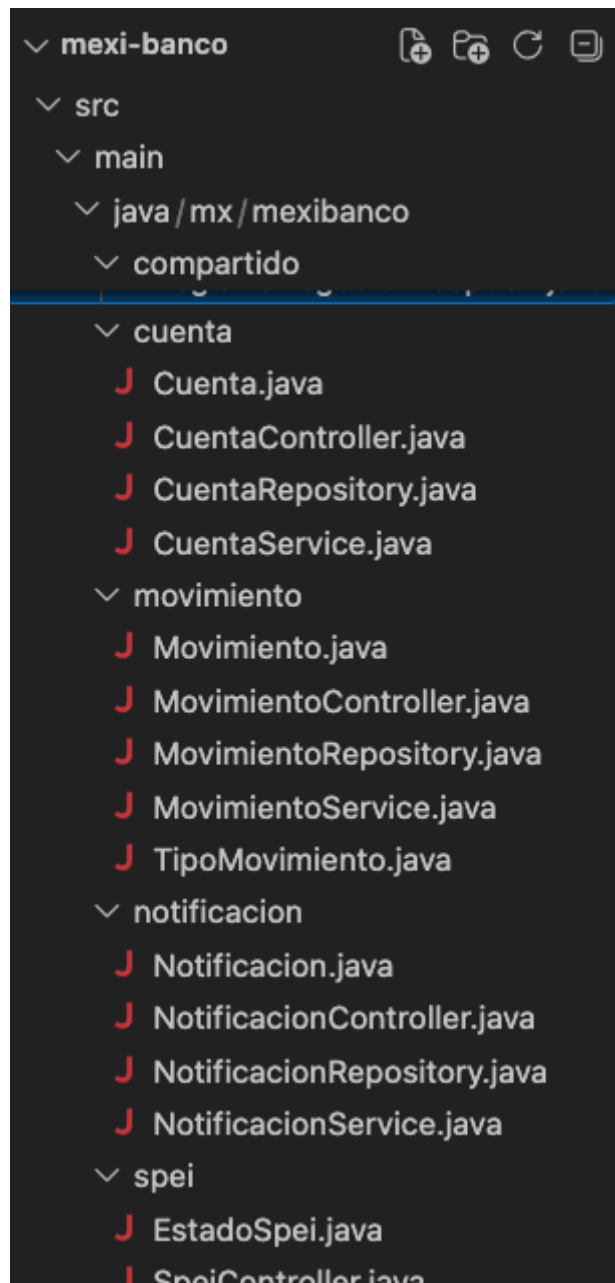


Paso 5

Para cada uno de los módulos cuenta, movimiento, transferencia, notificación y spei se tiene una arquitectura basada en la separación de responsabilidades en las siguientes capas:

- **Controller:** Es la capa que se encarga de recibir las peticiones externas, validar los parámetros de dichas entradas y devolver la respuesta al cliente.
- **Service:** Es la capa que contiene las reglas de negocio. El controller le pasa los datos a este servicio, el cual ejecuta las reglas de negocio, cálculos, validaciones complejas y coordina la información antes de guardarla o devolverla.
- **Repository:** Es la capa que se encarga de comunicarse con la base de datos. Ejecuta las consultas para crear, leer, actualizar o eliminar registros. El service llama al repositorio cuando necesita persistir o extraer información.

Los archivos restantes son las clases que definen qué atributos y funciones tiene un movimiento, una cuenta, una transferencia, una notificación y un spei. En los archivos restantes se encuentran excepciones y el manejador de errores.

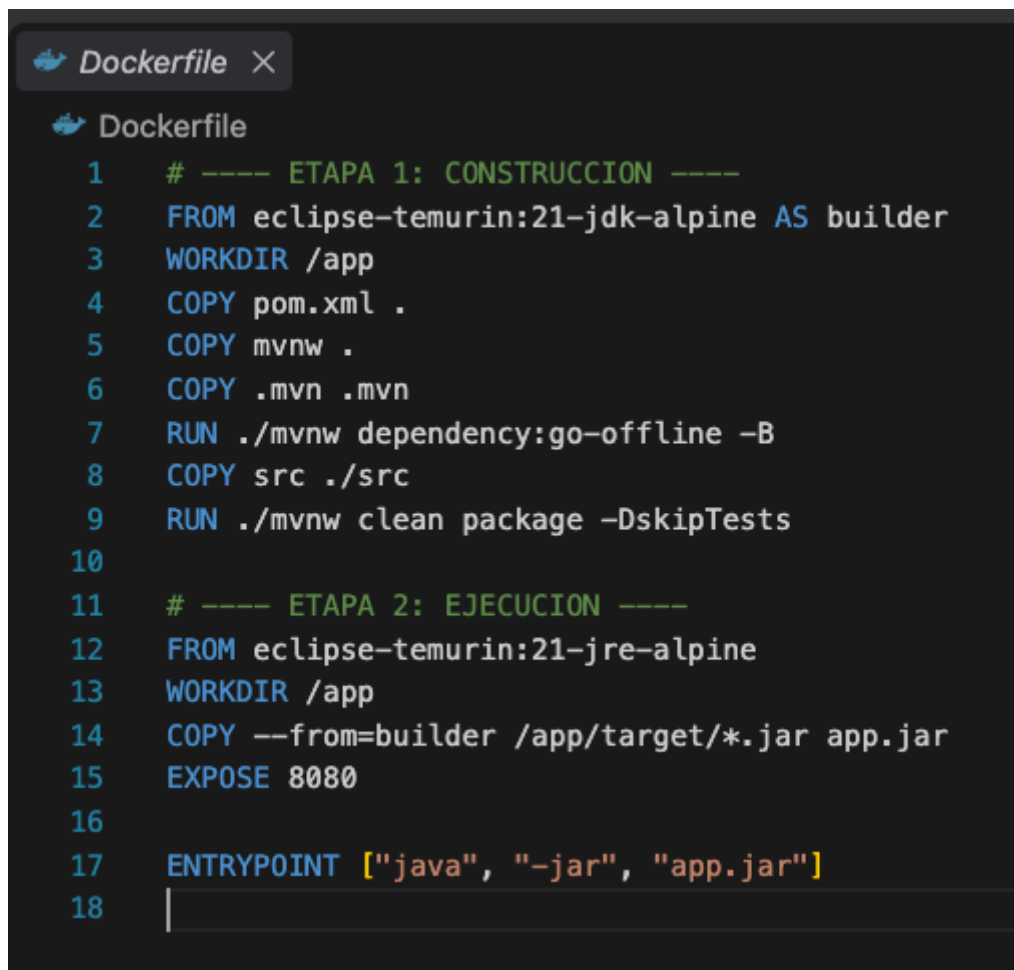


Paso 6

Dentro del Dockerfile se tiene lo siguiente:

1. FROM eclipse-temurin:21-jdk-alpine AS builder: Utiliza una imagen base de Java 21 que contiene el JDK completo necesario para compilar código. Usa la variante alpine y le asigna el alias builder.
2. WORKDIR /app: Crea el directorio /app dentro del contenedor y establece que todos los comandos posteriores se ejecuten allí.
3. COPY pom.xml ./ / COPY mvnw ./ / COPY .mvn .mvn: Copia únicamente los archivos de configuración de dependencias al contenedor.
4. RUN ./mvnw dependency:go-offline -B: Descarga todas las dependencias del proyecto al caché de Docker.
5. COPY src ./src: Copia el código fuente de la app.

6. RUN ./mvnw clean package -DskipTests: Compila el código y genera el archivo ejecutable .jar.
7. FROM eclipse-temurin:21-jre-alpine: Inicia una imagen completamente nueva partiendo de cero. Esta vez usa el JRE para ejecutar aplicaciones Java, no para compilarlas, por lo que pesa considerablemente menos y es más seguro.
8. WORKDIR /app: Establece el directorio de trabajo en esta nueva imagen.
9. COPY --from=builder /app/target/*.jar app.jar: Desde la imagen builder, extrae únicamente el archivo .jar ya compilado y lo pega en esta nueva imagen bajo el nombre estandarizado app.jar. Todo el código fuente, las dependencias temporales y las herramientas de Maven se destruyen.
10. EXPOSE 8080: Indica que la aplicación escuchará tráfico en el puerto 8080.
11. ENTRYPOINT ["java", "-jar", "app.jar"]: Define el comando exacto que se ejecutará en cuanto el contenedor arranque, levantando la app.



```
Dockerfile X
Dockerfile
1  # ---- ETAPA 1: CONSTRUCCION ----
2  FROM eclipse-temurin:21-jdk-alpine AS builder
3  WORKDIR /app
4  COPY pom.xml .
5  COPY mvnw .
6  COPY .mvn .mvn
7  RUN ./mvnw dependency:go-offline -B
8  COPY src ./src
9  RUN ./mvnw clean package -DskipTests
10
11 # ---- ETAPA 2: EJECUCION ----
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 COPY --from=builder /app/target/*.jar app.jar
15 EXPOSE 8080
16
17 ENTRYPOINT ["java", "-jar", "app.jar"]
18
```